# Manual de funcionamiento

**Equipo:** Arepas.exe

**Integrantes:**

Luis Alejandro Toloza Mendoza

Nicoll Navarro Simanca

Luz Marina Sanchez Rodriguez

Fecha: 10 de noviembre de 2026

# NetCheck Firewall Guard

# Manual de funcionamiento

Equipo Arepas.exe · Versión 1.1.0 · 10 de noviembre de 2026

Este manual describe cómo está construido NetCheck Firewall Guard por dentro: su arquitectura, el flujo de una validación, las reglas del inspector, el registro de auditoría, los controles de seguridad y las limitaciones. Está dirigido a quien necesita entender, mantener o documentar el sistema. Para ponerlo en marcha consulta el *Manual de instalación*; para operar el panel, el *Manual de uso*.

**Estado real del proyecto.** Un backend FastAPI inspecciona paquetes IPv4 de prueba generados desde el formulario del panel. Aún no se captura tráfico real ni se modifican reglas del firewall; el módulo con NetfilterQueue (solo Linux) es una fase futura.

## 1. Arquitectura

El sistema tiene dos capas: un panel web y una API que contiene toda la lógica de seguridad.

| Componente | Archivo | Función |
|---|---|---|
| Panel web | `frontend/index.html` | Muestra estadísticas, eventos y alertas; envía los datos del formulario |
| API | `backend/main.py` | Expone los endpoints, valida entradas, mantiene contadores y sirve el panel |
| Generador de paquetes | `core/packet_simulator.py` | Construye un paquete IPv4 de prueba con los datos del formulario |
| Inspector | `core/packet_inspector.py` | Valida la cabecera IPv4 byte a byte y recalcula el checksum |
| Motor de decisión | `core/decision_engine.py` | Convierte el estado de la inspección en PERMITIR o DESCARTAR |
| Auditoría | `audit/audit_logger.py` | Escribe y lee el registro `netcheck.log` |

## 2. Flujo de una validación

1. El usuario envía el formulario y el navegador llama a `POST /api/simulate`.
2. FastAPI y Pydantic validan los datos: IP IPv4 válida, tamaño de cabecera entre 20 y 60 en múltiplos de 4, y el resto de campos solo con valores permitidos. Si algo falla, responde 422.
3. `packet_simulator` construye los bytes de un paquete IPv4 con la IP, el tamaño y el defecto elegidos.
4. `packet_inspector` analiza esos bytes y devuelve un estado (VALID, INVALID o UNVERIFIED) con el motivo.
5. `decision_engine` traduce el estado a una acción: ALLOW o DROP.

6. `main.py` actualiza los contadores y la lista de eventos recientes, protegidos con un candado para que varios hilos no se pisen.
7. Si la acción es DROP, `audit_logger` escribe una línea en el log.
8. La API responde con el resultado, el motivo y las estadísticas, y el panel se refresca. Además, el panel consulta la API automáticamente cada 5 segundos.

Cuando exista captura real, el paso 3 se sustituirá por los bytes que entregue NetfilterQueue, y los pasos 4 a 8 seguirán igual.

## 3. Validaciones del inspector

El inspector lee los bytes directamente en lugar de usar un parser permisivo, para no «corregir» sin avisar un paquete malformado.

| Comprobación | Regla | Motivo si falla |
|---|---|---|
| Longitud mínima | Al menos 20 bytes | Paquete demasiado corto |
| Versión | Los 4 primeros bits deben valer 4 | Versión IP distinta de 4 |
| IHL | Tamaño de cabecera entre 20 y 60 bytes | Longitud de cabecera fuera de rango |
| Cabecera completa | Deben llegar todos los bytes de opciones | Cabecera truncada |
| Longitud total | No menor que la cabecera ni mayor que los bytes recibidos | Longitud total inconsistente |
| TTL | Distinto de 0 | TTL fuera de rango |
| Checksum | Recalculado según RFC 1071 | Checksum de cabecera incorrecto |

## 4. Cómo funciona el checksum

El checksum IPv4 es la suma, en palabras de 16 bits y con acarreo, de toda la cabecera, seguida de su complemento a uno. Si la cabecera llega íntegra, al volver a calcular la suma incluyendo el checksum recibido el resultado es 0. Cualquier valor distinto indica que la cabecera cambió o se corrompió.

**Integridad, no autenticidad.** Este mecanismo detecta errores accidentales. No prueba autenticidad: un atacante puede recalcular un checksum válido para un paquete manipulado. Por eso en los informes conviene hablar de integridad y no de autenticidad.

## 5. Estados y decisión

| Estado | Causa | Acción |
|---|---|---|
| VALID | Todas las comprobaciones pasaron | ALLOW |
| INVALID | Una o más anomalías detectadas | DROP |

| Estado | Causa | Acción |
|---|---|---|
| UNVERIFIED | Checksum no comprobado o excepción interna | DROP (política restrictiva) |

La política «ante la duda, descartar» es más segura, pero puede cortar tráfico legítimo si falla el inspector. Es una decisión de diseño que el equipo debe dejar documentada, porque el informe de la Clase 1 dice que ante errores el tráfico legítimo debe seguir fluyendo.

## 6. Endpoints de la API

| Método y ruta | Descripción |
|---|---|
| `GET /` | Entrega el panel web |
| `GET /docs` | Documentación interactiva de FastAPI |
| `GET /api/status` | Estado de la API y estadísticas |
| `GET /api/statistics` | Contadores: total, valid, dropped, pending |
| `GET /api/events` | Actividad reciente de la sesión, incluidos los paquetes válidos |
| `GET /api/alerts` | Últimos 50 descartes leídos del log |
| `POST /api/alerts/ack` | Pone en cero las alertas pendientes |
| `POST /api/simulate` | Genera, inspecciona y decide sobre un paquete de prueba |

Ejemplo de petición a `/api/simulate`:

```
{
  "interface":                                                    "eth0",
  "ip":                                                     "192.168.1.10",
  "headerLength":                                                     20,
  "checksum":                                                   "invalid",
  "issue":                                                        "none",
  "action":                                                       "DROP"
}
```

## 7. Registro de auditoría

- Solo se registran los paquetes **descartados**, como define la Clase 1.
- Cada línea incluye fecha y hora en UTC real, IP de origen, IP de destino, acción y motivo.
- No se guarda el contenido (payload) de los paquetes.
- Antes de escribir, los campos se limpian: se eliminan saltos de línea y el carácter `|`, y se limita su longitud, para impedir que alguien falsifique entradas en el log.
- La carpeta por defecto es `backend/logs`; puede cambiarse con la variable de entorno `NETCHECK_LOG_DIR`.
- Al leer, solo se cargan en memoria las últimas líneas pedidas.

## 8. Controles de seguridad implementados

| Control | Dónde | Norma citada en los informes |
|---|---|---|
| Validación estricta de entradas con tipos y rangos | Modelo Pydantic en `main.py` | OWASP ASVS V5.1 |
| Validación sintáctica de cabeceras y checksum | `packet_inspector.py` | OWASP ASVS V5.1 |
| Descarte de paquetes anómalos | `decision_engine.py` | ISO/IEC 27033, NIST SSDF |
| Registro de auditoría sin datos sensibles | `audit_logger.py` | OWASP ASVS V7.4, SSDF PW.9 |
| Escape de texto antes de mostrarlo en el panel | `index.html` | OWASP ASVS V5 (codificación de salida) |
| Servidor ligado a 127.0.0.1 | `main.py` | ISO/IEC 27033 |

Las referencias normativas provienen de los informes de las Clases 1 y 2. Confirma la numeración exacta de cada control en las fuentes oficiales antes de entregar.

## 9. Limitaciones conocidas

- **Tráfico simulado:** no hay captura real ni bloqueo real; las decisiones son simuladas.
- **Contadores en memoria:** se reinician al detener el servidor.
- **Sin autenticación en la API:** está pensada para uso local. No la expongas a una red sin añadir autenticación y HTTPS.
- **Solo IPv4:** IPv6 queda como mejora futura, como se declaró en la Clase 1.
- **Rendimiento:** es un prototipo en Python; no está diseñado para volúmenes de tráfico de producción.
- **Sin pruebas automáticas todavía:** conviene añadir pruebas unitarias del inspector y del generador de paquetes.

## 10. Próximos pasos sugeridos

- Implementar el módulo de intercepción con NetfilterQueue en Linux y conectarlo al inspector existente.
- Persistir los contadores, por ejemplo reconstruyéndolos desde el log o con un archivo SQLite.
- Añadir pruebas unitarias con pytest para cada regla del inspector.
- Documentar en el informe la política de UNVERIFIED y la diferencia entre integridad y autenticidad.